Digital Signal Processing

Assignment 1: Fourier Transform

Voice Analysis, Spectral Enhancement and an Aural Exciter

Patrick Connolly (2777107C)
Zabiallah Azar (2893681A)

1 Recordings and Method

Zabi spoke the same sentence "Real Time Terahertz Holographic Imaging" twice into an iPhone 17 Pro, once at 5 cm and once at 1 m from the microphone. Both files are uncompressed WAV at 44.1 kHz, 16-bit, with identical left and right channels and a peak of 98.9 % of full scale, so neither recording is clipped. The scripts keep one channel, normalise it to a peak of ± 1 and trim the digital silence that GarageBand adds onto the end, using NumPy's `trim_zeros` function, with argument `"b"` to only cut trailing zeros.

Table 1: Recording parameters after trimming.

	5 cm	1 m
FFT length N	107 398	114 257
Bin spacing f_s/N	0.41 Hz	0.39 Hz

The FFT of the whole recording is taken in one go, so the bin spacing is under 0.5 Hz, which is fine enough to separate vowel fundamentals a few hertz apart. The frequency of bin k is built by hand. For filtering, the upper half of the axis is folded back, so that the mirrored bins k and $N-k$ always get the same gain and the inverse FFT stays real:

$$f_k = \frac{k f_s}{N}, \qquad \tilde{f}_k = \min(f_k,\ f_s - f_k)\,, \qquad k = 0, \ldots, N-1. \tag{1}$$

Spectra are plotted from bin 1 to $N/2$ (DC cannot go on a log axis) in decibels relative to the input peak, $20\log_{10}(|X[k]|/\max|X|)$.

Every filter in Tasks 2 and 3 is a real gain G multiplied onto the FFT, followed by an IFFT. Each gain is built from a single linear ramp,

$$r_{f_0,\,f_1} = \min\left(\max\left(\frac{\tilde{f} - f_0}{f_1 - f_0},\, 0\right),\, 1\right), \tag{2}$$

which is 0 below f_0 and 1 above f_1. A high-pass is $r_{f_0,\,f_1}$, a low-pass is $1 - r_{f_0,\,f_1}$, and a band-pass is the product of the two. Our first Task 2 attempt used hard brick-wall edges, and the 2–5 kHz boost made the voice audibly ring. Replacing every edge with a ramp shortened the impulse response of the gain and removed the ringing, so ramps are used throughout.

2 Task 1: Signal Analysis

Time domain. The 5 cm waveform (fig. 1) has clearly separated syllables with near-silent gaps between them. At 1 m (fig. 2) the gaps are filled in.

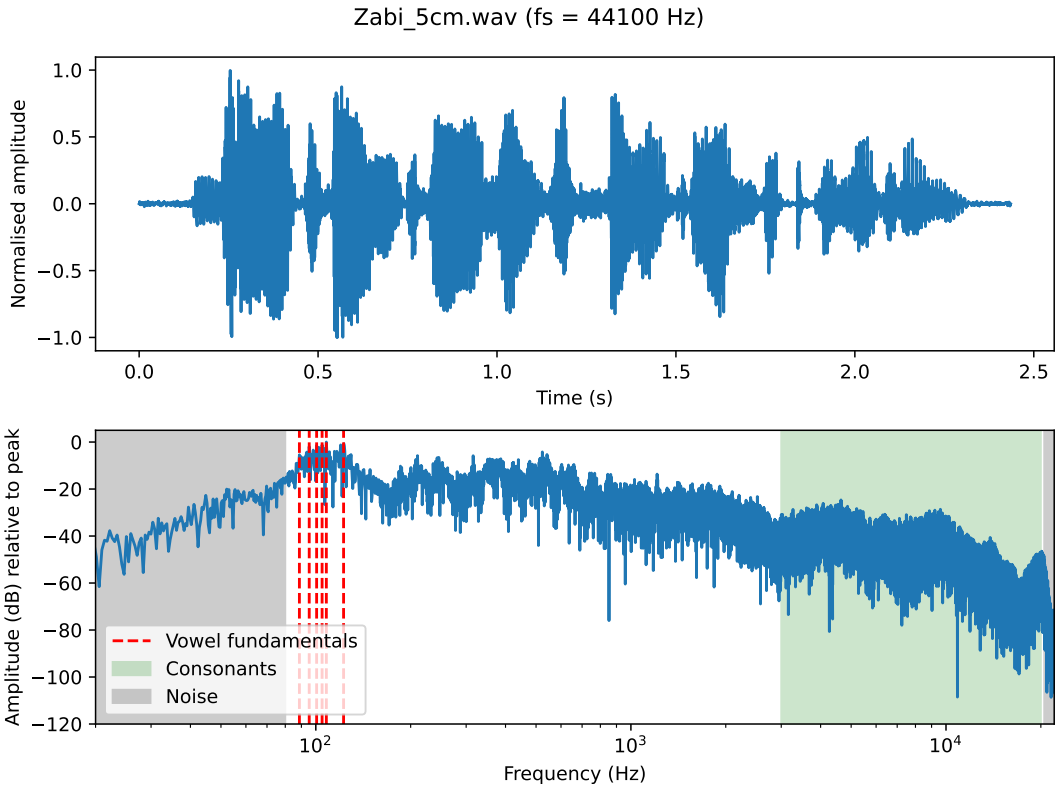


Figure 1: 5 cm recording: normalised waveform (top) and spectrum (bottom) with vowel fundamentals (red), consonant range (green) and noise regions (grey) marked.

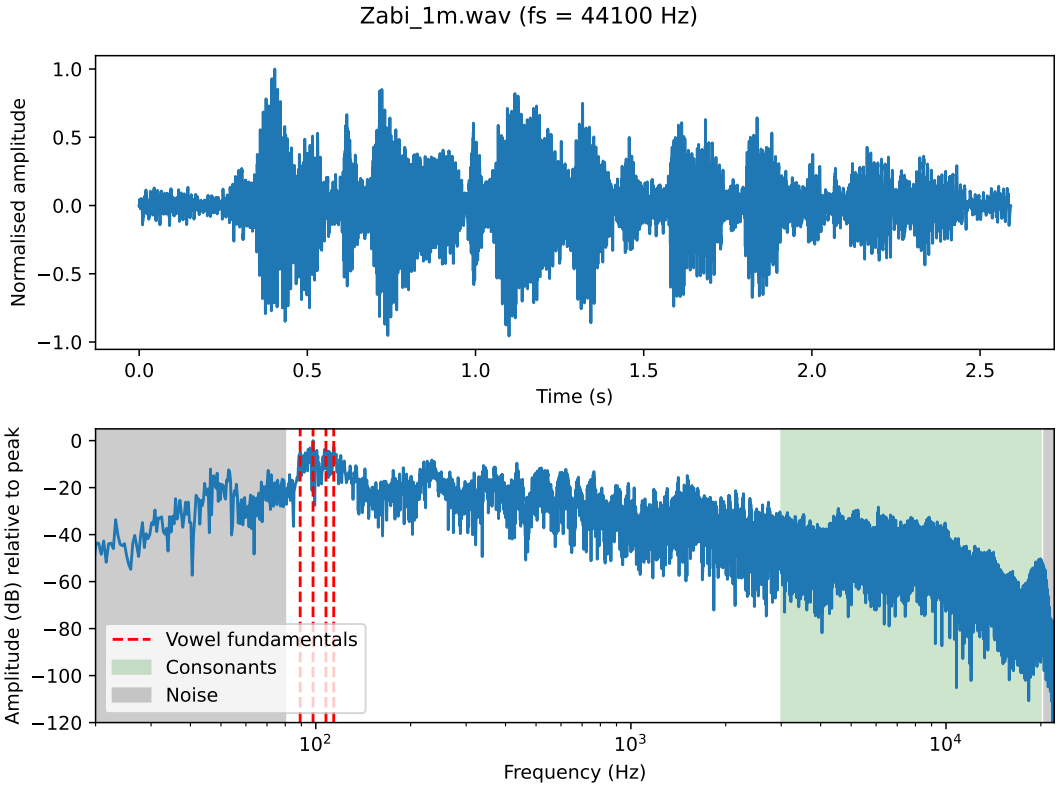


Figure 2: 1 m recording, marked in the same way as fig. 1.

Vowel fundamentals. Vowels are voiced, so each one produces a strong peak at the speaker's fundamental, with its harmonics at integer multiples above it. Zabi is a male so therefore, we looked for the first strong peaks above the noise region in the region of ≈ 85 Hz to 155 Hz. At 5 cm we marked six peaks (88.7 Hz, 95.3 Hz, 100.6 Hz, 104.7 Hz, 108 Hz and 122.5 Hz). Each vowel is spoken at a slightly different pitch, which is why they appear as separate peaks rather than one. At 1 m only four could be picked out (89.2 Hz, 98 Hz, 107.7 Hz and 114 Hz). The two recordings are separate takes, so the pitch of each vowel is slightly different, and at 1 m the room noise in the same range fills the gaps between neighbouring peaks, so some of them can no longer be told apart.

Consonants. Fricatives and plosives such as "s", "f" and "t" are bursts of turbulent air with no fundamental, so their energy sits high in the spectrum where the vowel harmonics have died away. Above ≈ 3 kHz the spectrum stops showing a regular harmonic comb and turns into a broad, noise-like plateau, so we start the consonant range there. It extends up to 20 kHz. The hump at 17 kHz to 20 kHz is still 16 dB to 24 dB above the silent lead-in, so it is speech, and above 20 kHz the spectrum falls off steeply due to the recorder's anti-aliasing filter which sits just below the Nyquist limit.

Noise. Two regions most likely contain only noise. Below 80 Hz is under the lowest vowel fundamental, so it cannot be voice. It holds room noise and, in the 5 cm recording, the low-frequency pops. Above 20.5 kHz is beyond human hearing and inside the recorder's anti-aliasing roll-off, so it carries no useful speech.

3 Task 2: Spectral Enhancement

Each recording is enhanced with a single gain curve G built from eq. (2):

$$Y[k] = G[k]\,X[k], \qquad y[n] = \mathrm{Re}\{\mathrm{IFFT}\{Y[k]\}\}\,, \tag{3}$$

$$G = \underbrace{r_{60,\,80}\,(1 - r_{20000,\,20500})}_{\text{noise cuts}}\ \underbrace{(1 + 0.3\,r_{1500,\,2000}\,(1 - r_{5000,\,6000}))}_{\text{presence, both files}}\ \underbrace{(1 + 0.4\,(1 - r_{120,\,150}))}_{\text{bass, 1 m only}} \tag{4}$$

The output is renormalised to a peak of 1 and saved as a 32-bit float WAV.

Table 2: Changes made to each recording in Task 2.

Change	Band	Applied to	Reason
Cut	below 80 Hz	both	rumble, and pops at 5 cm
Cut	above 20.5 kHz	both	noise region from Task 1
×1.3 (2.3 dB)	2 kHz to 5 kHz	both	presence and intelligibility
×1.4 (2.9 dB)	80 Hz to 120 Hz	1 m only	restore vowel fundamentals

5 cm: pops. The risk at close range is plosive pops, which appear below 80 Hz. The close recording also already has a strong low end, so no bass is added. Cutting below 80 Hz removes the pops without touching the voice, since the lowest fundamental is 88.7 Hz. The ramp starts at 60 Hz, so the cut is fully in place before the voice begins.

1 m: room noise and lost bass. At 1 m the voice is quieter relative to the room, and the direct bass of the voice is weaker. However, the low end of the 1 m spectrum is not lower than at 5 cm, because room noise fills it in. Boosting the whole bass region would mostly boost noise. The bass

boost is therefore limited to the band below 120 Hz, where the vowel fundamentals are strongest compared with the noise, and it fades back to unity by 150 Hz. Our first version boosted 100 Hz to 300 Hz by $\times 2$, but it missed two of the four fundamentals and made the silent parts noisier, so it was replaced.

Clarity. The 2 kHz to 5 kHz region holds the consonant detail that makes speech intelligible. It is lifted by 2.3 dB in both recordings. Figures 3 and 4 show the spectra before and after.

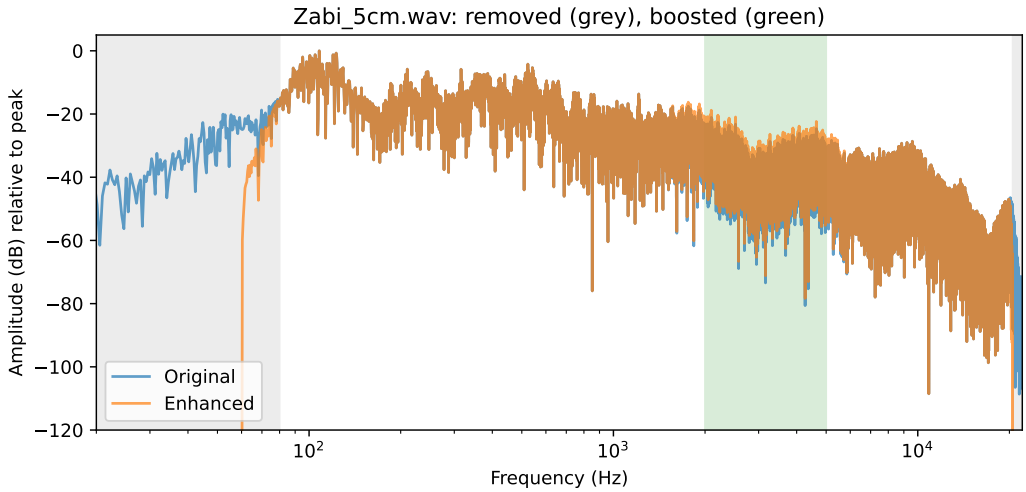


Figure 3: 5 cm recording before and after enhancement. Both curves use the original peak as 0 dB.

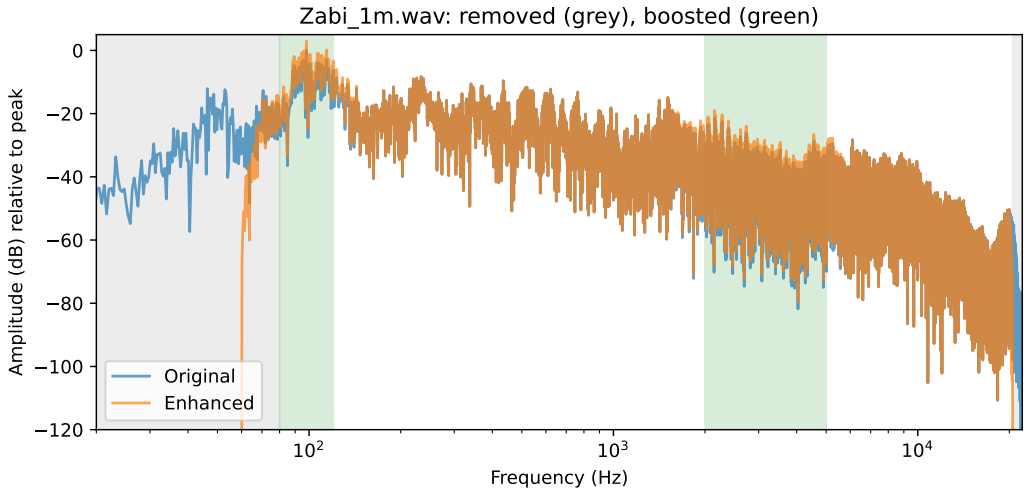


Figure 4: 1 m recording before and after enhancement, with the extra 80 Hz to 120 Hz boost.

4 Aural Exciter

We based our exciter on the Aphex block diagram from the brief and applied it to the 5 cm recording, after removing the pops and noise in the same way as in section 3. Like the diagram, it has two sidechains that are mixed back in with the original voice at the SUM block:

$$y[n] = x[n] + \text{mix} \cdot h[n] + \text{bass_mix} \cdot b[n], \tag{5}$$

where `mix` and `bass_mix` are the two Mix controls in the diagram and have the same names in our code.

In the Aural Exciter sidechain $h[n]$, the voice is high-passed and normalised to a peak of 1 before going through a tanh nonlinearity, as suggested in the brief:

$$d[n] = \tanh\big(\text{drive} \cdot x_{\text{hp}}[n]\big), \tag{6}$$

which creates harmonics of the voice. Because speech contains lots of frequencies at once, the tanh also creates sum and difference frequencies between them that can land anywhere in the spectrum, so we band-pass $d[n]$ again to keep only the harmonics we want, as the brief describes. The Big Bottom sidechain $b[n]$ works in a similar way on the low end, where the voice below 190 Hz is put through $\tanh(3x)$. This squashes loud bass more than quiet bass, so it acts like the dynamics processor in the diagram. All of the filters are done by multiplying the FFT and taking the IFFT, with sloped edges instead of hard cuts to avoid ringing.

Table 3: Exciter settings and how much each version changed the voice.

	v1 (first try)	v2 (final)
High-pass into tanh	800 Hz	3 kHz
Harmonics kept	all	4 kHz to 12 kHz
`drive`	5	5
`mix`	0.3	0.15
`bass_mix`	0	0.1
Change at 80 Hz to 190 Hz	0 dB	2.2 dB
Change at 4 kHz to 10 kHz	6.7 dB	2.8 dB
Change at 12 kHz to 20 kHz	7.5 dB	0 dB

Our first version (v1) boosted everything above 800 Hz by 6 dB to 9 dB (fig. 6), which made the voice sound harsh. Because we didn't filter after the tanh, it also put energy back below 80 Hz, where we had just removed the pops. For v2, we moved the high-pass up to 3 kHz and only kept harmonics between 4 kHz and 12 kHz. This band is centred around 6 kHz, where the brief says the ear is most sensitive. We also halved `mix` and added a small amount of Big Bottom bass. Figure 5 shows that v2 only adds content in those two bands. When listening back, v1 sounds much sharper than v2. This is most likely because v1 raises the 12 kHz to 20 kHz range by 7.5 dB, which is where the hissy "s" and "t" sounds sit, and keeps every harmonic the tanh creates at twice the mix level. v2 leaves that range untouched, so its added brightness stays in the region the ear is most sensitive to without making those sounds piercing.

One thing we noticed is that with `drive` at 5, the tanh barely bends the signal, since only about 1 % of samples are affected. This means most of v2's boost is really the existing 4 kHz to 12 kHz content turned up, with only some new harmonics on top. We tried a version with a drive of 20, which produced almost pure harmonics, but it sounded squawky, so we kept v2.

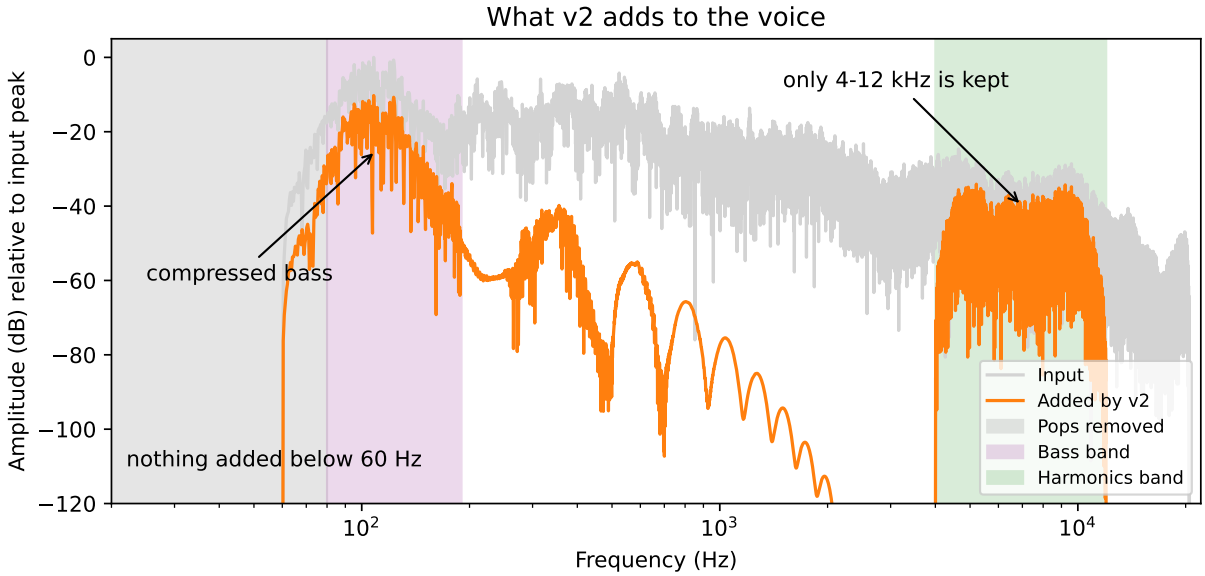


Figure 5: What exciter v2 adds to the voice, drawn over the input spectrum.

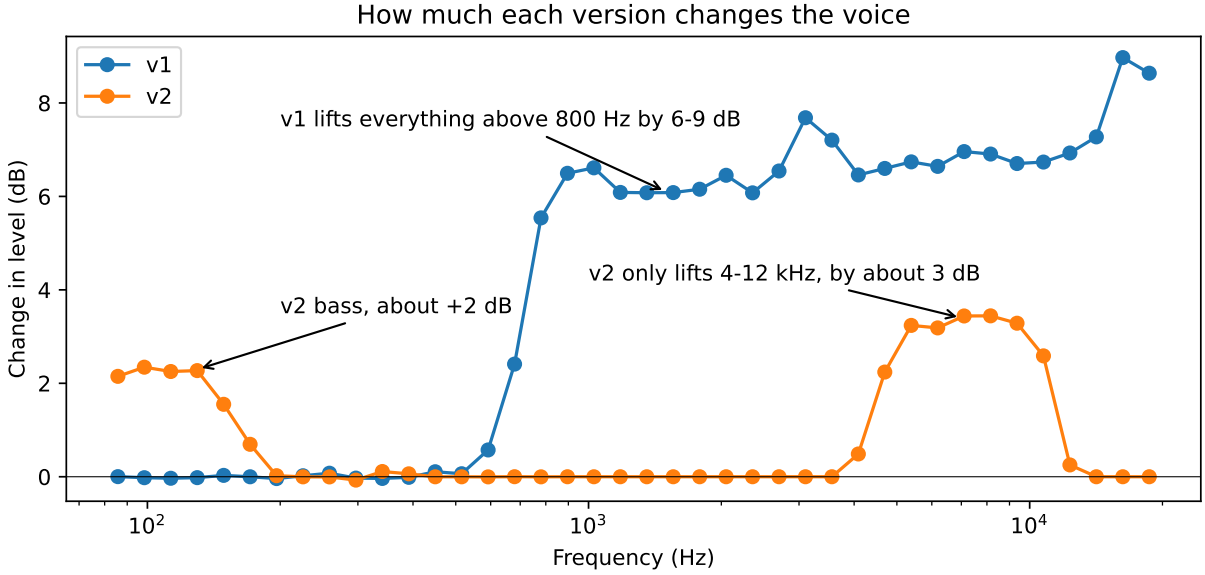


Figure 6: How much each version changes the voice level, averaged over 40 log-spaced bands.

5 Conclusion

Moving from 5 cm to 1 m raised the room-noise floor relative to the voice. Most of that noise is below 200 Hz, so it overlaps the vowel fundamentals and can only be partly removed with FFT gains. At close range the main problem was pops below 80 Hz, which could be removed cleanly. Ramped gain edges were essential, because hard edges made the enhanced voice ring. For the exciter, limiting both the input to the nonlinearity and the harmonics kept from it was what turned a harsh first attempt into a usable design. The final exciter lifts 4 kHz to 12 kHz by about 3 dB.

Appendix: Python Code

task1.py

```python
import os
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile

os.chdir(os.path.dirname(os.path.abspath(__file__))) # find the wav files wherever the script is run from

# Regions chosen by eye
# same speaker and sentence, so the consonants sit in the same range at both distances
marks = {
    "Zabi_5cm.wav": {"vowels": [88.7, 95.3, 100.6, 104.7, 108, 122.5], "consonants": (3000, 20000), "noise": [(20, 80), (20500, 22050)]},
    "Zabi_1m.wav": {"vowels": [89.2, 98, 107.7, 114], "consonants": (3000, 20000), "noise": [(20, 80), (20500, 22050)]},
}

# Main Loop
for filename, m in marks.items():
    fs, x = wavfile.read(filename)
    x = x[:, 0] # stereo file, both channels are the same
    x = x / np.max(np.abs(x)) # normalise to +-1
    x = np.trim_zeros(x, "b") # trim trailing zeros added by garageband ## "b" given in numpy doc just to cut trailing

    N = len(x)
    t = np.arange(N) / fs #for time axis

    # spectrum up to Nyquist, skipping the DC bin so the log axis works
    X = np.abs(np.fft.fft(x)[1:N // 2])
    f = np.arange(1, N // 2) * fs / N
    XdB = 20 * np.log10(X / np.max(X)) #dB relative to peak

    # plotting
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8, 6))
    fig.suptitle(f"{filename} (fs = {fs} Hz)")
    ax1.plot(t, x)
    ax1.set_xlabel("Time (s)")
    ax1.set_ylabel("Normalised amplitude")

    ax2.plot(f, XdB)
    ax2.set_xscale("log")
    ax2.set_xlim(20, fs / 2) # audible range up to Nyquist
    ax2.set_ylim(-120, 5)
    ax2.set_xlabel("Frequency (Hz)")
    ax2.set_ylabel("Amplitude (dB) relative to peak")
```

```
    ax2.vlines(m["vowels"], -120, 5, colors="r", linestyles="--", label="Vowel fundamentals")
    ax2.axvspan(*m["consonants"], color="g", alpha=0.2, label="Consonants")
    low, high = m["noise"]
    ax2.axvspan(*low, color="grey", alpha=0.4, label="Noise")
    ax2.axvspan(*high, color="grey", alpha=0.4)
    ax2.legend(loc="lower left")
    fig.tight_layout()
    fig.savefig(filename.replace(".wav", ".pdf"))

plt.show()
```

task2.py

```
import os
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile

os.chdir(os.path.dirname(os.path.abspath(__file__))) # find the wav files wherever the script is run from

filenames = ["Zabi_5cm.wav", "Zabi_1m.wav"]


# gain that goes from 0 at f0 up to 1 at f1, so 1 - ramp is a lowpass (same as task 3)
# slopes instead of hard steps because hard steps in the gain made the voice ring
# f is the frequency axis of the file being processed, set in the loop below
def ramp(f0, f1):
    return np.clip((f - f0) / (f1 - f0), 0, 1)


for filename in filenames:
    fs, x = wavfile.read(filename)
    x = x[:, 0] # stereo file, both channels are the same
    x = x / np.max(np.abs(x)) # normalise to +-1
    x = np.trim_zeros(x, "b") # trim trailing zeros added by garageband ## "b" given in numpy doc just to cut trailing
    N = len(x)

    #fft
    X = np.fft.fft(x)

    # frequency of each bin, the second half is a mirror of the first half
    # so fold it back so both halves get changed the same
    f = np.arange(N) * fs / N
    f = np.minimum(f, fs - f)

    # remove everything below 80 Hz (rumble, and pops for the 5cm one)
    # and the upper noise margin identified in task 1, above 20.5 kHz
    G = ramp(60, 80) * (1 - ramp(20000, 20500))

    # boost 2-5 kHz to make the voice clearer
    G *= 1 + 0.3 * ramp(1500, 2000) * (1 - ramp(5000, 6000)) # x1.3 is about +2.3 dB

    # boost bass for 1m only because bass is lost with distance
    # 5cm already has loads of bass (proximity effect)
    # only below 120 Hz where the vowel fundamentals are (89-114 Hz in task 1),
    # the room noise is worst compared to the voice at 120-200 Hz so that is left alone
    if filename == "Zabi_1m.wav":
        G *= 1 + 0.4 * (1 - ramp(120, 150)) # x1.4 is about +3 dB

    Y = X * G

```

```
    #ifft back to time domain
    y = np.real(np.fft.ifft(Y)) # imaginary part is basically 0
    y = y / np.max(np.abs(y)) # normalise again so it doesnt clip

    wavfile.write(filename.replace(".wav", "_enhanced.wav"), fs, y.astype(np.float32))

    #plot before and after
    half = N // 2
    mag_before = np.abs(X[1:half])
    mag_after = np.abs(Y[1:half])
    ref = np.max(mag_before) # same reference for both so the plot shows the real change
    dB_before = 20 * np.log10(mag_before / ref + 1e-12)
    dB_after = 20 * np.log10(mag_after / ref + 1e-12)

    fig, ax = plt.subplots(figsize=(8, 4))
    ax.plot(f[1:half], dB_before, label="Original", alpha=0.7)
    ax.plot(f[1:half], dB_after, label="Enhanced", alpha=0.7)

    # shade the bands that were changed, grey = removed, green = boosted
    ax.axvspan(20, 80, color="grey", alpha=0.15)
    ax.axvspan(20500, fs / 2, color="grey", alpha=0.15)
    if filename == "Zabi_1m.wav":
        ax.axvspan(80, 120, color="green", alpha=0.15)
    ax.axvspan(2000, 5000, color="green", alpha=0.15)

    ax.set_xscale("log")
    ax.set_xlim(20, fs / 2)
    ax.set_ylim(-120, 5)
    ax.set_xlabel("Frequency (Hz)")
    ax.set_ylabel("Amplitude (dB) relative to peak")
    ax.set_title(filename + ": removed (grey), boosted (green)")
    ax.legend(loc="lower left")
    fig.tight_layout()
    fig.savefig(filename.replace(".wav", "_task2.pdf"))

plt.show()
```

task3.py

```
import os
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile

# aural exciter based on the aphex block diagram in the brief
# exciter part: highpass -> tanh makes harmonics -> only keep some of them -> mix back in
# big bottom part: lowpass -> tanh squashes it like a compressor -> mix back in
# all the filtering is done by multiplying the fft and then doing the ifft (like task 2)

os.chdir(os.path.dirname(os.path.abspath(__file__))) # find the wav files wherever the script is
    run from

fs, x = wavfile.read("Zabi_5cm.wav")
x = x[:, 0] # stereo file, both channels are the same
x = x / np.max(np.abs(x)) # normalise to +-1
x = np.trim_zeros(x, "b") # trim trailing zeros added by garageband
N = len(x)

# frequency of each bin, folded like in task 2 so both halves get changed the same
f = np.arange(N) * fs / N
f = np.minimum(f, fs - f)


```

```
# gain that goes from 0 at f0 up to 1 at f1, so 1 - ramp is a lowpass
# used a slope instead of a hard cut because hard cuts can cause ringing
def ramp(f0, f1):
    return np.clip((f - f0) / (f1 - f0), 0, 1)


#fft
X = np.fft.fft(x)

# remove pops below 80 Hz and noise above 20.5 kHz first, same cuts as task 2
# task 2's 2-5 kHz boost is left out, making the voice clearer is the exciter's job here
X = X * ramp(60, 80) * (1 - ramp(20000, 20500))
x = np.real(np.fft.ifft(X))

# big bottom, lowpass then tanh squashes the loud bits like a compressor
# tanh puts a bit back below 80 Hz, so the pop cut is done again afterwards
# it's the same for every version so it only gets worked out once
low = np.real(np.fft.ifft(X * (1 - ramp(150, 190))))
low = low / np.max(np.abs(low))
bass = np.real(np.fft.ifft(np.fft.fft(np.tanh(3 * low)) * ramp(60, 80)))


# hp = highpass gain, band = which harmonics to keep, drive = how hard tanh gets pushed,
# mix and bass_mix = how much of each part gets added back
# returns only the part that gets added to the voice
def exciter(hp, band, drive, mix, bass_mix):
    # highpass, then normalise so drive does the same thing every time
    side = np.real(np.fft.ifft(X * hp))
    side = side / np.max(np.abs(side))

    # tanh makes the harmonics, its symmetric so only odd ones (3rd, 5th...)
    harm = np.tanh(drive * side)

    # only keep the harmonics inside the band
    harm = np.real(np.fft.ifft(np.fft.fft(harm) * band))
    harm = harm / np.max(np.abs(harm))

    return mix * harm + bass_mix * bass


# v1 first try: highpass at 800 Hz and keep all the harmonics (band = 1)
added1 = exciter(ramp(600, 800), 1, 5, 0.3, 0)

# v2: v1 boosts everything above 800 Hz (12-20 kHz goes up about 7 dB) and puts stuff back
# below 80 Hz where i'd removed the pops (see figure 1)
# so the highpass is moved up to 3 kHz, only harmonics from 4-12 kHz are kept,
# plus a bit of bass to try the big bottom side of the aphex diagram
hp2 = ramp(2000, 3000)
band2 = ramp(4000, 5000) * (1 - ramp(10000, 12000)) # highpass * lowpass = bandpass
added2 = exciter(hp2, band2, 5, 0.15, 0.1)

y1 = x + added1
y2 = x + added2

# save them so i can listen, normalised so they dont clip
out1 = y1 / np.max(np.abs(y1))
out2 = y2 / np.max(np.abs(y2))
wavfile.write("Zabi_5cm_exciter_v1_initial.wav", fs, out1.astype(np.float32))
wavfile.write("Zabi_5cm_exciter_v2_final.wav", fs, out2.astype(np.float32))

#plotting
half = N // 2
fp = f[1:half] # skip DC for the log axis
ref = np.max(np.abs(X[1:half])) # 0 dB = input peak so all the curves can be compared
```

```
88
89
90  def dB(s):
91      mag = np.abs(np.fft.fft(s)[1:half])
92      return 20 * np.log10(mag / ref + 1e-12)
93
94
95  x_dB = dB(x)
96
97  # figures 1 and 2: what each version adds, drawn over the input so you can see how loud it is
98  def plot_added(added, version, colour):
99      fig, ax = plt.subplots(figsize=(8, 4))
100     ax.plot(fp, x_dB, label="Input", color="lightgrey")
101     ax.plot(fp, dB(added), label=f"Added by {version}", color=colour)
102     ax.axvspan(20, 80, color="grey", alpha=0.2, label="Pops removed")
103     ax.set_xscale("log")
104     ax.set_xlim(20, fs / 2)
105     ax.set_ylim(-120, 5)
106     ax.set_xlabel("Frequency (Hz)")
107     ax.set_ylabel("Amplitude (dB) relative to input peak")
108     ax.set_title(f"What {version} adds to the voice")
109     return fig, ax
110
111
112 arrow = dict(arrowstyle="->")
113
114 fig1, ax1 = plot_added(added1, "v1", "C0")
115 ax1.annotate("almost as loud as the voice\nabove 800 Hz, sounds harsh", xy=(8000, -30), xytext
        =(1000, -95), arrowprops=arrow)
116 ax1.annotate("energy put back\nwhere the pops were", xy=(45, -50), xytext=(22, -100), arrowprops=
        arrow)
117 ax1.legend(loc="upper right", fontsize="small")
118
119 fig2, ax2 = plot_added(added2, "v2", "C1")
120 ax2.axvspan(80, 190, color="purple", alpha=0.15, label="Bass band")
121 ax2.axvspan(4000, 12000, color="green", alpha=0.15, label="Harmonics band")
122 ax2.annotate("only 4-12 kHz is kept", xy=(7000, -40), xytext=(1500, -8), arrowprops=arrow)
123 ax2.annotate("compressed bass", xy=(110, -25), xytext=(25, -60), arrowprops=arrow)
124 ax2.text(22, -110, "nothing added below 60 Hz")
125 ax2.legend(loc="lower right", fontsize="small")
126
127 # figure 3: how much each version changes the level of the voice, averaged over 40 bands
128 # so the curves are smooth enough to compare (starts at 80 Hz, below that the input is cut)
129 edges = np.geomspace(80, 20000, 41)
130 centres = np.sqrt(edges[:-1] * edges[1:]) # middle of each band on a log axis
131
132
133 def band_power(s):
134     P = np.abs(np.fft.fft(s)[1:half]) ** 2
135     return np.array([P[(fp >= lo) & (fp < hi)].mean() for lo, hi in zip(edges[:-1], edges[1:])])
136
137
138 Px = band_power(x)
139 fig3, ax3 = plt.subplots(figsize=(8, 4))
140 ax3.plot(centres, 10 * np.log10(band_power(y1) / Px), "o-", label="v1")
141 ax3.plot(centres, 10 * np.log10(band_power(y2) / Px), "o-", label="v2")
142 ax3.axhline(0, color="k", linewidth=0.5)
143 ax3.annotate("v1 lifts everything above 800 Hz by 6-9 dB", xy=(1500, 6.1), xytext=(200, 7.5),
        arrowprops=arrow)
144 ax3.annotate("v2 only lifts 4-12 kHz, by about 3 dB", xy=(7000, 3.4), xytext=(1000, 4.2),
        arrowprops=arrow)
145 ax3.annotate("v2 bass, about +2 dB", xy=(130, 2.3), xytext=(200, 3.5), arrowprops=arrow)
146 ax3.set_xscale("log")
147 ax3.set_xlabel("Frequency (Hz)")
```

```
ax3.set_ylabel("Change in level (dB)")
ax3.set_title("How much each version changes the voice")
ax3.legend(loc="upper left")

for fig in [fig1, fig2, fig3]:
    fig.tight_layout()
fig1.savefig("task3_v1.pdf")
fig2.savefig("task3_v2.pdf")
fig3.savefig("task3_change.pdf")
plt.show()
```